



Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics

Benjamin Bichsel
ETH Zurich, Switzerland
benjamin.bichsel@inf.ethz.ch

Timon Gehr
ETH Zurich, Switzerland
timon.gehr@inf.ethz.ch

Maximilian Baader
ETH Zurich, Switzerland
mbaader@inf.ethz.ch

Martin Vechev
ETH Zurich, Switzerland
martin.vechev@inf.ethz.ch

Abstract

Existing quantum languages force the programmer to work at a low level of abstraction leading to unintuitive and cluttered code. A fundamental reason is that dropping temporary values from the program state requires explicitly applying quantum operations that safely *uncompute* these values.

We present Silq, the first quantum language that addresses this challenge by supporting safe, automatic uncomputation. This enables an intuitive semantics that implicitly drops temporary values, as in classical computation. To ensure physicality of Silq's semantics, its type system leverages novel annotations to reject unphysical programs.

Our experimental evaluation demonstrates that Silq programs are not only easier to read and write, but also significantly shorter than equivalent programs in other quantum languages (on average -46% for Q#, -38% for Quipper), while using only half the number of quantum primitives.

CCS Concepts: • **Software and its engineering** → *Formal language definitions; Language features*; • **Computer systems organization** → *Quantum computing*.

Keywords: Quantum Language, Uncomputation, Semantics

ACM Reference Format:

Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '20)*, June 15–20, 2020, London, UK. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3385412.3386007>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PLDI '20, June 15–20, 2020, London, UK

© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-7613-6/20/06...\$15.00
<https://doi.org/10.1145/3385412.3386007>

1 Introduction

Quantum algorithms leverage the principles of quantum mechanics to achieve an advantage over classical algorithms. In recent years, researchers have continued proposing increasingly complex quantum algorithms [5, 9, 12, 13, 24, 33], driving the need for expressive, high-level quantum languages.

The Need for Uncomputation. Analogously to the classical setting, quantum computations often produce temporary values. However, as a key challenge specific to quantum computation, removing such values from consideration induces an *implicit measurement* collapsing the state [21, §4.4]. In turn, collapsing can result in unintended side-effects on the state due to the phenomenon of *entanglement*. Surprisingly, due to the quantum principle of *deferred measurement* [21, §4.4], preserving values until computation ends is equivalent to measuring them immediately after their last use, and hence cannot prevent this problem.

To remove temporary values from consideration without inducing an implicit measurement, algorithms in existing languages must explicitly *uncompute* all temporary values, i.e., modify their state to enable ignoring them without side-effects. This results in a significant gap from quantum to classical languages, where discarding temporary values typically requires no action (except for heap values not garbage-collected). This gap is a major roadblock preventing the adoption of quantum languages as the implicit side-effects resulting from uncomputation mistakes, such as silently dropping temporary values, are highly unintuitive.

This Work. We present Silq, a high-level quantum language which bridges this gap by automatically uncomputing temporary values. To this end, Silq's type system exploits a fundamental pattern in quantum algorithms, stating that uncomputation can be done safely if (i) the original evaluation of the uncomputed value can be described classically, and (ii) the variables used to evaluate it are preserved and can thus be leveraged for uncomputation.

As uncomputation happens behind the scenes and is always safe, Silq is the first quantum language to provide *intuitive semantics*: if a program type-checks, its semantics

<pre> 1 d := a b c; 2 1 with_computed (OR a b) \$ 2 \t -> OR t c </pre>	Silq Quipper	<pre> 1 using(t=Qubit()){ 2 OR(a,b,t); 3 OR(t,c,d); 4 Adjoint OR(a,b,t); 5 } </pre> Q#
--	---	--

Figure 1. Benefit of Silq’s automatic uncomputation.

<pre> 1 cTri := 0:int[rrbar]; 2 for j in [0..rrbar] { 3 for k in [j+1..rrbar] { 4 if ee[tau[j]][tau[k]] 5 && ew[j] && ew[k]{ 6 cTri += 1; 7 } } } </pre>	Silq Quipper QWire, 4-6
--	--

Figure 2. Comparing Silq to Quipper and QWire code, more readable version in App. A.

follows an intuitive recipe that simply drops temporary values. Importantly, Silq’s semantics is *physical*, i.e., can be realized on a quantum random access machine (QRAM) [11].

Overall, Silq allows expressing quantum algorithms more safely and concisely than existing quantum programming languages, while typically using only half the number of quantum primitives. In our evaluation (§8), we show that across 28 challenges from recent coding contests [15, 16], Silq programs require on average 46% less code than Q# [30]. Similarly, expressing the triangle finding algorithm [26] in Silq requires 38% less code than Quipper [7].

Main Contributions. Our main contributions are:

- Silq¹, a high-level quantum language enabling safe, automatic uncomputation (§4).
- A full formalization of Silq’s key language fragment Silq-core (§5), including its type system (§6) and semantics (§7), whose physicality relies on its type system.
- An implemented type-checker², proof-of-concept simulator², and development environment³ for Silq.
- An evaluation, showing that Silq code is more concise and readable than code in existing languages (§8).

Future Benefits. We expect Silq to advance various tools central to programming quantum computers. First, Silq compilers may require fewer qubits, e.g., by more flexibly picking the time of uncomputation. Second, static analyzers for Silq can safely assume (instead of explicitly proving) temporary values are discarded safely. Third, Silq simulators may improve performance by dropping temporary values instead of explicitly simulating uncomputation. Finally, Silq’s key language features are relevant beyond Silq, as they can be incorporated into existing languages such as QWire or Q#.

¹<http://silq.ethz.ch/>

²<https://github.com/eth-sri/silq/tree/pldi2020>

³<https://marketplace.visualstudio.com/items?itemName=eth-sri.vscode-silq>

2 Benefit of Automatic Uncomputation

Next, we show the benefit of automatic uncomputation compared to explicit uncomputation in existing languages, including Q# [30], Quipper [7], and QWire [22].

Explicit Uncomputation. Fig. 1 shows code snippets which compute the OR of three qubits. This is easily expressed in Silq (top left), which leverages automatic uncomputation for $a || b$. In contrast, Q# (right) requires (i) allocating a new qubit t initialized to 0 in Line 1, (ii) using OR^4 to store the result of $a || b$ in t in Line 2, (iii) using OR to store the result of $t || c$ in the pre-allocated qubit d in Line 3, and (iv) uncomputing t by reversing the operation from Line 2 in Line 4. Here, $Adjoint\ OR$ is the inverse of OR and thus resets t to its original value of 0. Hence, the *implicit measurement* induced by removing t from consideration in Line 5 always measures the value 0, which has no side-effects (see §3). We note that we cannot allocate t within OR , as Q# enforces that qubits must be deallocated in the function that allocates them.

Explicit uncomputation is even more tedious in Fig. 2, which shows part of a triangle finding algorithm originally encoded⁵ by the authors of Quipper [7] (middle). The condition is easily expressed in Silq (left, Lines 4–5) using nested expressions. In contrast, the equivalent Quipper code is obfuscated by uncomputation of sub-expressions.

Convenience Functions. As uncomputation is a common task, various quantum languages try to reduce its boilerplate code by introducing convenience functions such as `ApplyWith` in Q#. Fig. 1 shows a Quipper implementation using a similar function `with_computed`, which (i) evaluates $a || b$ in Line 1, (ii) uses the result t to compute $t || c$ in Line 2, and (iii) implicitly uncomputes t . However, this still requires explicitly triggering uncomputation using `with_computed` and introducing a name t for the result of $a || b$. In particular, this does not enable a natural nesting of expressions, as the sub-expression $OR\ a\ b$ needs to be managed by `with_computed`. Moreover, `with_computed` cannot ensure safety: we can make the uncomputation unsafe by flipping the bit stored in b between Line 1 and Line 2, triggering an implicit measurement.

Non-Linear Type Systems. Most quantum languages cannot ensure that all temporary values are safely uncomputed for a fundamental reason: they support reference sharing in a non-linear type system and hence cannot statically detect when values are removed from consideration (which happens when the last reference to the value goes out of scope). Besides Quipper, which we discuss in more detail, there are many other works of this flavor, including LIQuiD [32], ProjectQ [29], Cirq [31], and QisKit [1].

⁴Since Q# does not support OR natively, we would need to implement it too.

⁵Taken from: <https://www.mathstat.dal.ca/~selinger/quipper/doc/src/Algorithms/TF/QWTFP.html#line-494>

Linear Type Systems. Other languages, like QPL [27] and QWire, introduce a linear type system to prevent accidentally removing values from consideration, which corresponds to not using a value. However, linear type systems still require explicit uncomputation that ends in *assertive termination* [7]: the programmer must (manually) assert that uncomputation correctly resets temporary values to 0. Most recently, ReQWire [23] introduced syntactic conditions sufficient to verify assertive termination. However, ReQWire can only verify explicitly provided uncomputation (except for purely classical oracle functions, see below), and cannot statically reason across function boundaries as its type system does not address uncomputation — a key contribution of Silq.

Further, linear type systems introduce significant syntactic overhead for constant (i.e., read-only) variables where enforcing linearity is not necessary. Fig. 2 demonstrates this in QWire code (right), where encoding only Lines 4–6 from Silq (left) requires 19 lines of code, even when we generously assume built-in primitives and omit parts of the required type annotations. We note that while QWire [22] does not explicitly claim to be high-level, we are not aware of more high-level quantum languages that achieve a level of safety similar to QWire — even though it cannot prevent implicit measurement caused by incorrect manual uncomputation.

In contrast, Silq uses a linear type system to detect values removed from consideration (which are automatically uncomputed), but reduces notational overhead by treating constant variables non-linearly.

Bennett’s Construction. Various languages, like Quipper, ReVerC [2], and ReQWire, support Bennett’s construction [3], which can lift purely classical (oracle) functions to quantum inputs, automatically uncomputing all temporary values computed in the function. Concretely, this standard approach (i) lifts all primitive classical operations in the oracle function to quantum operations, (ii) evaluates the function while preserving all temporary values, (iii) uses the function’s result, and (iv) uncomputes temporary values by reversing step (ii). Bennett’s construction is also supported by Qumquat⁶, which skips step (i) above by annotating quantum functions as @qq.garbage and calling them with notation analogous to Quipper’s with_computed.

However, Bennett’s construction is unsafe when the oracle function contains quantum operations: as we demonstrate in App. C, it can fail to drop temporary values without side-effects. In contrast, Silq safely uncomputes temporary values in functions containing quantum operations.

Importantly, Silq’s workflow when defining oracle functions is different from existing languages: while the latter typically require programmers to define a purely classical oracle function and then apply Bennett’s construction, Silq

programmers can define oracle functions directly using primitive quantum operations, implicitly relying on Silq’s automatic uncomputation.

Summary. In contrast to other languages, Silq (i) enables intuitive yet physical semantics and (ii) statically prevents errors that are not detected in existing languages, while (iii) avoiding the notational overhead associated with languages that achieve (less) static safety (e.g., QWire).

3 Background on Quantum Computation

We now provide a short review of the core concepts in quantum computation relevant to this work.

Qubit. The state of a quantum bit (qubit) is a superposition (linear combination) $\varphi = \gamma_0 |0\rangle + \gamma_1 |1\rangle$, where $\gamma_0, \gamma_1 \in \mathbb{C}$, and $\|\varphi\|^2 = \|\gamma\|^2 = \|\gamma_0\|^2 + \|\gamma_1\|^2$ denotes the probability of being in state φ . In particular, we allow $\|\varphi\| < 1$ to indicate that a measurement yields state φ with probability $\|\varphi\|$ — a common convention [27, Convention 3.3].

Hilbert Space, Ground Set, Basis State. More generally, assume a variable on a classical computer can take on values from a finite *ground set* S . Then, the quantum states induced by S form the Hilbert space $\mathcal{H}(S)$ consisting of the formal complex linear combinations [25, p. 379] over S :

$$\mathcal{H}(S) := \left\{ \sum_{v \in S} \gamma_v |v\rangle \mid \gamma_v \in \mathbb{C} \right\}.$$

Here, each element $v \in S$ corresponds to a (*computational*) *basis state* $|v\rangle$. For $S = \{0, 1\}$, we obtain the Hilbert space of a single qubit $\mathcal{H}(\{0, 1\}) = \{\gamma_0 |0\rangle + \gamma_1 |1\rangle \mid \gamma_0, \gamma_1 \in \mathbb{C}\}$, with computation basis states $|0\rangle$ and $|1\rangle$.

We note that we use the (standard) inner product $\langle \cdot | \cdot \rangle$ throughout this work, defined by

$$\left\langle \sum_{v \in S} \gamma_v |v\rangle \mid \sum_{v \in S} \gamma'_v |v\rangle \right\rangle = \sum_{v \in S} \gamma_v \gamma'_v.$$

Tensor Product. A system of multiple qubits can be described using the tensor product $\otimes$. For example, for two qubits $\varphi_0 = |0\rangle$ and $\varphi_1 = \frac{1}{\sqrt{2}} |0\rangle - i \frac{1}{\sqrt{2}} |1\rangle$, the composite state is $\varphi_0 \otimes \varphi_1 = \frac{1}{\sqrt{2}} |0\rangle \otimes |0\rangle - i \frac{1}{\sqrt{2}} |0\rangle \otimes |1\rangle = \frac{1}{\sqrt{2}} |0\rangle |0\rangle - i \frac{1}{\sqrt{2}} |0\rangle |1\rangle$. Here, we first used the linearity of $\otimes$ in its first argument and then omitted $\otimes$ for convenience. Simplifying notation further, we may also write $|0\rangle |0\rangle$ as $|0, 0\rangle$.

Entanglement. A composite state is called *entangled* if it cannot be written as a tensor product of single qubit states, but needs to be written as a sum of tensor products. For example, the above composite state $\varphi_0 \otimes \varphi_1$ is unentangled, while $\Phi^+ = \frac{1}{\sqrt{2}} |0\rangle |0\rangle + \frac{1}{\sqrt{2}} |1\rangle |1\rangle$ is entangled.

Measurement. To acquire information about a quantum state, we can (partially) measure it. Measurement has a probabilistic nature; if we measure $\varphi = \sum_{v \in S} \gamma_v |v\rangle$, we obtain

⁶ Available at <https://github.com/patrickrall/Qumquat>, commit 27d6794

the value $v' \in S$ with probability $\|\gamma_{v'}\|^2$. As a fundamental law of quantum mechanics, if we measure the value v' , the state after the measurement is $\gamma_{v'} |v'\rangle$ (we do not normalize this state to preserve linearity). This is referred to as the *collapse* of φ to $\gamma_{v'} |v'\rangle$, since superposition is lost.

Importantly, measuring part of a state can affect the whole state. To illustrate the effect of measuring the first part $|v\rangle$ of $\sum_{v,w} \gamma_{v,w} |v\rangle \otimes |w\rangle$, we first rewrite it to $\sum_v \gamma_v |v\rangle \otimes \tilde{\varphi}_v$, separating out the remainder $\tilde{\varphi}_v$ of the state, where $\|\tilde{\varphi}_v\| = 1$. This is a common technique and always possible for appropriate choices of γ_v and $\tilde{\varphi}_v$. Then, measuring the first part to be v' yields state $\gamma_{v'} |v'\rangle \otimes \tilde{\varphi}_{v'}$, also collapsing the remainder of the state.

Linear Isometries. Besides measurements, we can also manipulate quantum states using *linear isometries*, i.e., linear functions $f: \mathcal{H}(S) \rightarrow \mathcal{H}(S')$ preserving inner products: for all $\varphi, \varphi' \in \mathcal{H}(S)$, $\langle f(\varphi) | f(\varphi') \rangle = \langle \varphi | \varphi' \rangle$. Linear isometries generalize the commonly used notion of unitary operations, which additionally require that vector spaces $\mathcal{H}(S)$ and $\mathcal{H}(S')$ have the same dimension. As this prevents dynamically allocating and deallocating qubits⁷, we use the more general notion of linear isometries in this work.

QRAM. As a computational model for quantum computers, this work assumes a quantum random access machine (QRAM) [11]. A QRAM consists of a classical computer extended with quantum storage supporting state preparation, some unitary gates, and measurement. QRAMs can be extended to support linear isometries, by (i) padding input and output space to have the same dimension (using state preparation) and (ii) approximating the resulting unitary operation arbitrarily well using a standard set of universal quantum gates [21, §4.5.3].

No-Cloning. The no-cloning theorem states that cloning an arbitrary quantum state is unphysical: we cannot achieve the operation $\varphi \mapsto \varphi \otimes \varphi$. Silq's type system prevents cloning.

4 Overview of Silq

We now illustrate Silq on Grover's algorithm, a widely known quantum search algorithm [8], [21, §6.1]. It can be applied to any NP problem, where finding the solution may be hard, but verification of a solution is easy.

Fig. 3 shows a Silq implementation of grover. Its input is an oracle function f from (quantum) unsigned integers represented with n qubits to (quantum) booleans, mapping all but one input w^* to 0. Here, Silq uses the generic parameter n to parametrize the input type `uint[n]` of f . Then, `grover` outputs an n -bit unsigned integer w which is equal to w^* with high probability.

⁷Since given input space $\mathcal{H}(S)$, allocating qubits leads to a larger output space $\mathcal{H}(S')$, requiring $\mathcal{H}(S)$ and $\mathcal{H}(S')$ to have the same dimension (as enforced by unitary operations) prevents dynamically allocating qubits.

4.1 Silq Annotations

Classical Types. The first argument of `grover` is a *generic parameter* n , used to parametrize f . It has type $!\mathbb{N}$, which indicates classical natural numbers of arbitrary size. Here, an annotation $!$ indicates n is classically known, i.e., it is in a basis state (not in superposition), and we can manipulate it classically. For example, θ has type $!\mathbb{B}$. In contrast, $\mathbf{H}(0)$ applies Hadamard $\mathbf{H}$ (defined shortly) to θ and yields $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Thus, $\mathbf{H}(0)$ is of type $\mathbb{B}$ and not of (classical) type $!\mathbb{B}$.

In general, we can liberally use classical variables like normal variables on a classical computer: we can use them multiple times, or drop them. We also annotate parameter f as classical, writing the annotation as $\tau! \rightarrow \tau'$ instead of $!\tau \rightarrow \tau'$ to avoid the ambiguity between $!(\tau \rightarrow \tau')$ and $(!\tau) \rightarrow \tau'$.⁸

Qfree Functions. The type of f is annotated as **qfree**, which indicates the semantics of f can be described classically: we can capture the semantics of a **qfree** function g as a function $\bar{g}: S \rightarrow S'$ for ground sets S and S' . Note that since S' is a ground set, $\bar{g}$ can never output superpositions. Then, g acting on $\sum_{v \in S} \gamma_v |v\rangle$ yields $\sum_{v \in S} \gamma_v |\bar{g}(v)\rangle$, where for simplicity $\sum_{v \in S} \gamma_v |v\rangle$ does not consider other qubits untouched by g .

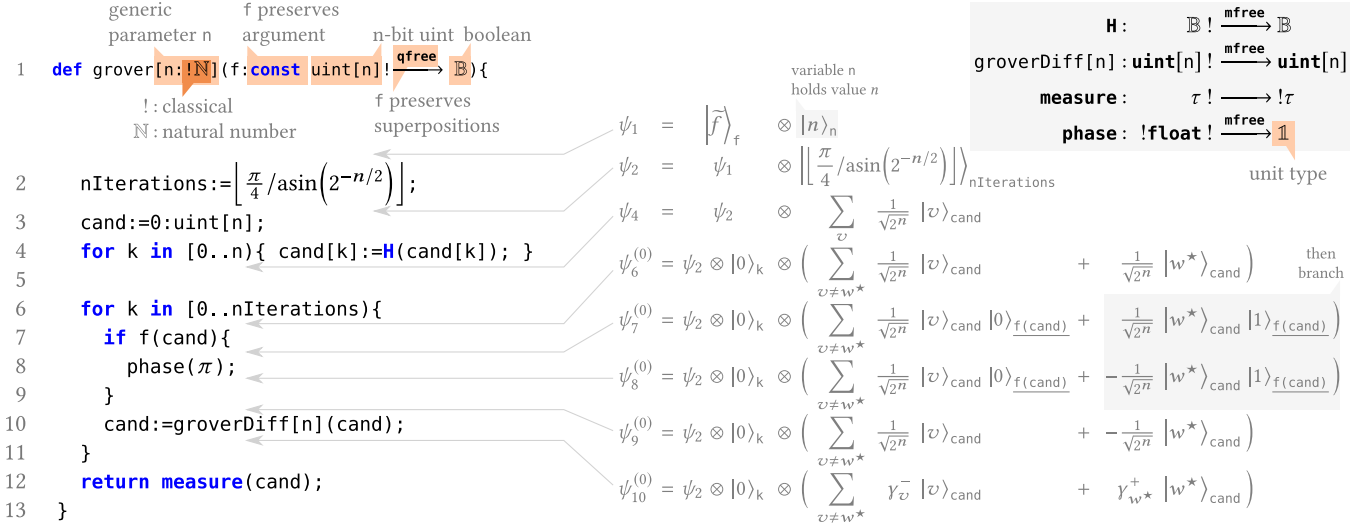
For example, the **qfree** function $\mathbf{X}$ flips the bit of its input, mapping $\sum_{v=0}^1 \gamma_v |v\rangle$ to $\sum_{v=0}^1 \gamma_v |\bar{\mathbf{X}}(v)\rangle$, for $\bar{\mathbf{X}}(v) = 1 - v$. In contrast, the Hadamard transform $\mathbf{H}$ maps $\sum_{v=0}^1 \gamma_v |v\rangle$ to $\sum_{v=0}^1 \gamma_v \frac{1}{\sqrt{2}}(|0\rangle + (-1)^v |1\rangle)$. As this semantics cannot be described by a function on ground sets, $\mathbf{H}$ is not **qfree**.

Constant Parameters. Note that $\mathbf{X}$ (introduced above) transforms its input — it does not preserve it. In contrast, the parameter of f is annotated as **const**, indicating f preserves its input, i.e., treats it as a read-only control. Thus, running f on $\sum_{v \in S} \gamma_v |v\rangle$ yields $\sum_{v \in S} \gamma_v |v\rangle \otimes \varphi_v$, where φ_v follows the semantics of f . Because f is also **qfree**, $|v\rangle \otimes \varphi_v = |\bar{f}(v)\rangle$ for some $\bar{f}: S \rightarrow S \times S'$. Combining both, we conclude that $\bar{f}(v) = (v, \tilde{f}(v))$ for some function $\tilde{f}: S \rightarrow S'$.

An example of a possible instantiation of f is `NOT`, which maps $\gamma_0 |0\rangle + \gamma_1 |1\rangle$ to $\gamma_0 |0, 1\rangle + \gamma_1 |1, 0\rangle$. Here, `NOT`: $\{0, 1\} \rightarrow \{0, 1\}$ maps $v \mapsto 1 - v$ and `NOT`: $\{0, 1\} \rightarrow \{0, 1\} \times \{0, 1\}$ maps $v \mapsto (v, 1 - v) = (v, \text{NOT}(v))$.

Function parameters not annotated as **const** are not accessible after calling the function — the function *consumes* them. For example, `groverDiff` consumes its argument (see top-right box in Fig. 3). Hence, the call in Line 10 consumes `cand`, transforms it, and writes the result into a new variable with the same name `cand`. Similarly, `measure` in Line 12 consumes `cand` by measuring it.

⁸Annotating functions as classical indicates that their function bodies are classically known (at runtime). We note that classical functions can still perform quantum operations: for example, $\mathbf{H}: \mathbb{B}! \xrightarrow{\text{mfree}} \mathbb{B}$ is classical, meaning that the quantum operations performed by $\mathbf{H}$ are classically known.



Lifted Functions. We introduce the term **lifted** to describe `qfree` functions with exclusively `const` parameters, as such functions are crucial for uncomputation. In particular, we could write the type of `f` as `uint[n]  $\xrightarrow{\text{lifted}}$  B`.

4.2 Silq Semantics

Next, we discuss the semantics of Silq on `grover`.

Input State. In Fig. 3, the state of the system after Line 1 is ψ_1 , where the state of `f: uint[n]!  $\xrightarrow{\text{qfree}}$  B` is described as a function $\tilde{f}: \{0, \dots, 2^n - 1\} \rightarrow \{0, 1\}$. We note that later, our formal semantics represents the state of functions as Silq-core expressions (§7). However, as the semantics of `f` can be captured by $\tilde{f}$, this distinction is irrelevant here. Next, Line 2 initializes the classical variable `nIterations`, yielding ψ_2 .

Superpositions. Lines 3–4 result in state ψ_4 , where `cand` holds the equal superposition of all n -bit unsigned integers. To this end, Line 4 updates the k^{th} bit of `cand` by applying the Hadamard transform `H` to it.

Loops. The loop in Line 6 runs `nIterations` times. Each loop iteration increases the coefficient of $|w^*\rangle$, thus increasing the probability of measuring w^* in Line 12. We now discuss the first loop iteration ($k = 0$). It starts from state $\psi_6^{(0)}$ which introduces variable `k`. For convenience, $\psi_6^{(0)}$ splits the superposition into w^* and all other values.

Conditionals. Intuitively, Lines 7–9 flip the sign of those coefficients for which `f(cand)` returns true. To this end, we first evaluate `f(cand)` and place the result in a temporary variable `f(cand)`, yielding state $\psi_7^{(0)}$. Here and in the following, we write $\underline{e}$ for a temporary variable that contains the result of evaluating e . Then, we determine those summands of

$\psi_7^{(0)}$ where `f(cand)` is true (marked as “then branch” in Fig. 3), and run `phase( $\pi$ )` on them. This yields $\psi_8^{(0)}$, as `phase( $\pi$ )` flips the sign of coefficients. Lastly, we drop `f(cand)` from the state, yielding $\psi_9^{(0)}$.

Grover’s Diffusion Operator. Completing the explanations of our example, Line 10 applies Grover’s diffusion operator to `cand`. Its implementation consists of 6 lines of code (see App. B). It increases the weight of solution w^* , obtaining $\|y_{w^*}^+\| > \|\frac{1}{\sqrt{2^n}}\|$, and decreases the weight of non-solutions $v \neq w^*$, obtaining $\|y_v^-\| < \|\frac{1}{\sqrt{2^n}}\|$. After one loop iteration, this results in state $\psi_{10}^{(0)}$. Repeated iterations of the loop in Lines 6–11 further increase the coefficient of w^* , until it is approximately 1. Thus, measuring `cand` in Line 12 returns w^* with high probability.

4.3 Uncomputation

While dropping the temporary value `f(cand)` from $\psi_8^{(0)}$ is intuitive, achieving this physically requires uncomputation.

Without uncomputation, simply removing `f(cand)` from consideration in Line 9 would induce an implicit measurement.⁹ Concretely, measuring and dropping `f(cand)` would collapse $\psi_8^{(0)}$ to one of the following two states (ignoring `f`, `n`, and `k`):

$$\psi_8^{(0,0)} = \sum_{v \neq w^*} \frac{1}{\sqrt{2^n}} |v\rangle_{\text{cand}} \quad \text{or} \quad \psi_8^{(0,1)} = -\frac{1}{\sqrt{2^n}} |w^*\rangle_{\text{cand}}.$$

In this case, as the probability of obtaining $\psi_8^{(0,1)}$ is only $\frac{1}{2^n}$, `grover` returns the correct result w^* with probability $\frac{1}{2^n}$, i.e., it degrades to random guessing.

⁹Formally, this corresponds to taking the partial trace over `f(cand)`.

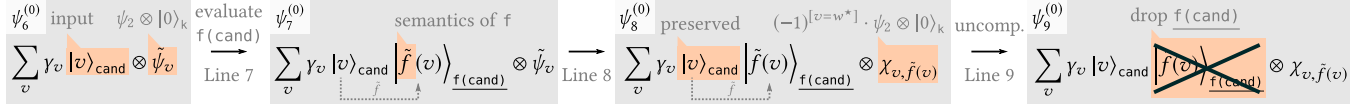


Figure 4. Uncomputation of $f(\text{cand})$ is safe. The term $(-1)^{[v=w^*]}$ equals -1 if $v = w^*$ and 1 otherwise.

```

def useConsumed(x:ℬ){
  y := H(x); // consumes x
  return (x,y);
} // undefined identifier x

def useConsumedFixed(const x:ℬ){
  // \psi_1 = \sum_{v=0}^1 \gamma_v |v\rangle_x
  // \psi_2 = \sum_{v=0}^1 \gamma_v |v\rangle_x \otimes |v\rangle_{\underline{x}}
  y := H(x);
  // \psi_3 = \sum_{v=0}^1 \gamma_v |v\rangle_x \otimes (|0\rangle_y + (-1)^v |1\rangle_y)
  return (x,y);
}

def discard[n:ℕ](x:uint[n]){
  y := x % 2; // '%' supports quantum inputs
  return y;
} // parameter 'x' is not consumed

def nonQfree(const x:ℬ, y:ℬ){
  if H(x) { y := X(y); }
  return y;
} // non-lifted quantum expression must be consumed

def nonConst(c:ℬ){
  if X(c) { phase(\pi); } // X consumes c
} // non-lifted quantum expression must be consumed

def nonConstFixed(const c:ℬ){
  // \psi_1 = \sum_{v=0}^1 \gamma_v |v\rangle_c
  if X(c) { phase(\pi); }
  // \psi_2 = \sum_{v=0}^1 (-1)^{1-v} \gamma_v |v\rangle_c
}

def condMeas(const c:ℬ, x:ℬ){
  if c { x := measure(x); }
} // cannot call function
// 'measure[ℬ]' in 'mfree' context

def revMeas(){
  return reverse(measure);
} // reversed function must be mfree

```

Figure 5. Examples of invalid Silq programs, their error messages, and possible fixes (where applicable).

Without correct intervention from the programmer, all existing quantum languages would induce an implicit measurement in Line 9, or reject grover. This is unfortunate as grover cleanly and concisely captures the programmer’s intent. In contrast, Silq achieves the intuitive semantics of dropping $f(\text{cand})$ from $\psi_8^{(0)}$, using uncomputation. In general, uncomputing x is possible whenever in every summand of the state, the value of x can be reconstructed (i.e., determined) from all other values in this summand. Then, reversing this reconstruction removes x from the state.

Automatic Uncomputation. To ensure that uncomputing $f(\text{cand})$ is possible, the type system of Silq ensures that $f(\text{cand})$ is **lifted**, i.e., (i) f is **qfree** and (ii) cand is **const**: it is preserved until uncomputation in Line 9.

Fig. 4 illustrates why this is sufficient. Evaluating f in Line 7 adds a temporary variable $f(\text{cand})$ to the state, whose value can be computed from cand using $\tilde{f}$ (as f is **qfree** and cand is **const**). Then, Line 8 transforms the remainder $\tilde{\psi}_v$ of the state to $\chi_{v, \tilde{f}(v)}$. The exact effect of Line 8 on the state is irrelevant for uncomputation, as long as it preserves cand , ensuring we can still reconstruct $f(\text{cand})$ from cand in $\psi_8^{(0)}$. Thus, reversing the operations of this reconstruction (i.e., reversing f) uncomputes $f(\text{cand})$ and yields $\psi_9^{(0)}$.

4.4 Preventing Errors: Rejecting Invalid Programs

Fig. 5 demonstrates how the type system of Silq rejects invalid programs. We note that the presented examples are not exhaustive — we discuss additional challenges in §6.

Error: Using Consumed Variables. In `useConsumed`, `H` consumes `x` and stores its result in `y`. Then, it accesses `x`, which leads to a type error as `x` is no longer available.

Assuming we want to preserve `x`, we can fix this code by marking `x` as **const** (see `useConsumedFixed`). Then, instead of consuming `x` in the call to `H` (which is disallowed as `x` must be preserved), Silq implicitly duplicates `x`, resulting in ψ_2 , and then only consumes the duplicate $\underline{x}$.

Implicit Duplication. It is always safe to implicitly duplicate constant variables, as such duplicates can be uncomputed (in `useConsumedFixed`, uncomputation is not necessary as the duplicate is consumed). In contrast, it is typically impossible to uncompute duplicates of consumed quantum variables, which may not be available for uncomputation later. Hence, Silq treats constant variables non-linearly (they can be duplicated or ignored), but treats non-constant variables linearly (they must be used exactly once).

We note that duplication $\sum_v \gamma_v |v\rangle \mapsto \sum_v \gamma_v |v\rangle |v\rangle$ is physical and can be implemented using CNOT, unlike the unphysical cloning $\sum_v \gamma_v |v\rangle \mapsto (\sum_v \gamma_v |v\rangle) \otimes (\sum_v \gamma_v |v\rangle) = \sum_{v,w} \gamma_v \gamma_w |v\rangle |w\rangle$ discussed earlier.

Error: Discarding Variables. Function `discard` does not annotate `x` as **const**, meaning that its callers expect it to consume `x`. However, the body of `discard` does not consume `x`, hence calling `discard` would silently discard `x`. As the callee does not know if `x` can be uncomputed, Silq rejects this code. A possible fix is annotating `x` as **const**, which would be in line with preserving `x` in the function body.

Error: Uncomputation Without Qfree. Silq rejects the function `nonQfree`, as $\mathbf{H}(x)$ is not **lifted** (since $\mathbf{H}$ is not **qfree**), and hence its result cannot be automatically uncomputed. Indeed, automatic uncomputation of $\mathbf{H}(x)$ is not possible in this case, intuitively because $\mathbf{H}$ introduces additional entanglement preventing uncomputation in the end. We provide a more detailed mathematical derivation of this subtle fact in App. C. To prevent this case, Silq only supports uncomputing **qfree** expressions.

We note that because x is **const** in `nonQfree`, $\mathbf{H}$ does not consume it, but a duplicate of x .

Error: Uncomputation Without Const. Silq rejects the function `nonConst`, as $\mathbf{X}(c)$ is not **lifted** (since it consumes c). Indeed, automatic uncomputation is not possible in this case, as the original value of c is not available for uncomputation of $\mathbf{X}(c)$. To get this code to type-check, we can mark c as **const** (see `nonConstFixed`) to clarify that c should remain in the context. Then, Silq automatically duplicates c before calling $\mathbf{X}$, which thus consumes a duplicate of c , leaving the original c available for later uncomputation.

Temporary Constants. In contrast to `nonConst`, which consumes c , `grover` does not consume cand in Line 7 (Fig. 3), even though cand is not annotated as **const** either. This is because Silq temporarily annotates cand as **const** in `grover`. In general, Silq allows temporarily annotating some variables as **const** for the duration of a statement or a consumed subexpression. Our implementation determines which variables to annotate as **const** as follows: If a variable is encountered in a position where it is not expected to be **const** (as in $\mathbf{X}(c)$), it is consumed, and therefore any further occurrence of that variable will result in an error (whether **const** or not). If a variable is encountered in a position where it is expected to be **const** (as in $f(\text{cand})$), we temporarily mark it as **const** until the innermost enclosing statement or consumed subexpression finishes type checking.

Mfree. Silq’s main advantage over existing quantum languages is its safe, automatic uncomputation, enabled by its novel annotations **const** and **qfree**. To ensure all Silq programs are physical (i.e., can be physically realized on a QRAM), we leverage one additional annotation **mfree**, indicating a function does not perform measurements. This allows us to detect (and thus prevent) attempts to reverse measurements and to apply measurements conditioned on quantum values.

Error: Conditional Measurement. Silq rejects `condMeas`, as it applies a measurement conditioned on quantum variable c . This is not realizable on a QRAM, as the `then`-branch requires a physical action and we cannot determine whether or not we need to carry out the physical action without measuring the condition. However, changing the type of c to $!\mathbb{B}$ would fix this error, as conditional measurement is possible if c is classical. We note that Silq could also detect this error if

measurement was hidden in a function passed to `condMeas`, as this function would not be **mfree**. Here, it is crucial that Silq disallows implicit measurement — otherwise, it would be hard to determine which functions are **mfree**.

Reverse. Silq additionally also supports reversing functions, where expression `reverse(f)` returns the inverse of function f . In general, all quantum operations except measurement describe linear isometries (see §3) and are thus injective. Hence, if f is also surjective (and thus bijective), we can reverse it, meaning `reverse(f)` is well-defined on all its inputs.

Reverse Returns Unsafe Functions. When f is not surjective, `reverse(f)` is only well-defined on the range of f . Hence, it is the programmer’s responsibility to ensure reversed functions never operate on invalid inputs.

For example, $y := \text{dup}(x)$ duplicates x , mapping $\sum_v \gamma_v |v\rangle_x$ to $\sum_v \gamma_v |v\rangle_x |v\rangle_y$. Thus, `reverse(dup)(x, y)` operates on states $\sum_v \gamma_v |v\rangle_x |v\rangle_y \otimes \tilde{\psi}_v$, for which it yields $\sum_v \gamma_v |v\rangle_x \otimes \tilde{\psi}_v$, uncomputing y . On other states, `reverse(dup)` is undefined. As `reverse(dup)` is generally useful for (unsafe) uncomputation, we introduce its (unsafe) shorthand **forget**.

When realizing a reversed function on a QRAM, the resulting program is defined on all inputs but only behaves correctly on valid inputs. For example, we can implement `reverse(dup)(x, y)` by running `if x { y := X(y); }` and discarding y , which has unintended side-effects (due to implicit measurement) unless originally $x=y$.

Error: Reversing Measurement. Silq rejects `revMeas` as it tries to reverse a measurement, which is physically impossible according to the laws of quantum mechanics. Thus, `reverse` only operates on **mfree** functions.

Discussion: Annotations as Negated Effects. We can view annotations **mfree** and **qfree** as indicating the absence of effects: **mfree** indicates a function does not perform a measurement, while **qfree** indicates the function does not introduce quantum superposition. As we will see later, all **qfree** functions in Silq are also **mfree**.

5 The Silq-Core Language Fragment

In this section, we present the language fragment Silq-core of Silq, including syntax (§5.1) and types (§5.2).

Silq-core is selected to contain Silq’s key features, in particular all its annotations. Compared to Silq, Silq-core omits features (such as the imperative fragment and dependent types) that distract from its key insights. We note that in our implementation, we type-check and simulate full Silq.

5.1 Syntax of Silq-Core

Fig. 6 summarizes the syntax of Silq-core.

Expressions. Silq-core expressions include constants and built-in functions (c), variables (x), measurement (**measure**),

$e ::= c \mid x \mid \text{measure} \mid \text{reverse} \mid \text{if } e \text{ then } e_1 \text{ else } e_2 \mid$ $e'(e_1, \dots, e_n) \mid \lambda(\beta_1 x_1 : \tau_1, \dots, \beta_n x_n : \tau_n).e$	
expressions	$(\vec{e})$ $(\vec{\beta}\vec{x} : \vec{\tau})$
types	$\tau ::= \mathbb{1} \mid \mathbb{B} \mid \bigotimes_{k=1}^n \tau_k \mid \bigotimes_{k=1}^n \beta_k \tau_k \mid \xrightarrow{\alpha} \tau' \mid !\tau$
annotations	$\alpha \subseteq \{\mathbf{mfree}, \mathbf{qfree}\}$ $\beta \subseteq \{\mathbf{const}\}$

Figure 6. Syntax, types, and annotations.

$$\Gamma ::= \beta_1 x_1 : \tau_1, \dots, \beta_n x_n : \tau_n \quad \Gamma \vdash^\alpha e : \tau$$

Figure 7. Typing judgments.

and reversing quantum operations (**reverse**). Further, its if-then-else construct **if** e **then** e_1 **else** e_2 is syntactically standard, but supports both classical ($!\mathbb{B}$) and quantum ($\mathbb{B}$) condition e . Function application $e'(\vec{e})$ explicitly takes multiple arguments. Likewise, lambda abstraction $\lambda(\vec{\beta}\vec{x} : \vec{\tau}).e$ describes a function with multiple parameters $\{x_i\}_{i=1}^n$ of types $\{\tau_i\}_{i=1}^n$, annotated by $\{\beta_i\}_{i=1}^n$, as discussed in §5.2 (next).

We note that Silq-core can support tupling as a built-in function c .

Universality. Assuming built-in functions c include **X** (enabling CNOT by **if** $x \{y := X(y)\}$) and arbitrary operations on single qubits (e.g., enabled by **rotX**, **rotY**, and **rotZ**), Silq-core is *universal for quantum computation*, i.e., it can approximate any quantum operation to arbitrary accuracy [21].

5.2 Types and Annotations of Silq-Core

Further, Fig. 6 introduces the types τ of Silq-core.

Primitive Types. Silq-core types include standard primitive types, including $\mathbb{1}$, the singleton type that only contains the element “()”, and $\mathbb{B}$, the Boolean type describing a single qubit. We note that it is straightforward to add other primitive types like integers or floats to Silq-core.

Products and Functions. Silq-core also supports products, where we often write $\tau_1 \times \dots \times \tau_n$ for $\bigotimes_{k=1}^n \tau_k$, and functions, where $!$ emphasizes that functions are classically known (i.e., we do not discuss superpositions of functions). Function parameters and functions themselves may be annotated by β_i and α , respectively, as discussed shortly. As usual, $\times$ binds stronger than $\rightarrow$.

Finally, Silq-core supports annotating types as classical.

Annotations. Fig. 6 also lists all Silq-core annotations.

Our annotations express restrictions on the computations of Silq-core expressions and functions, ensuring the physicality of its programs. For example, for quantum variable $x : \mathbb{B}$, the expression **if** x **then** $f(0)$ **else** $f(1)$ is only physical if f is **mfree** (note that x does not appear in the two branches).

6 Typing Rules

In this section, we introduce the typing rules of Silq. Most importantly, they ensure that every sub-expression that is not consumed can be uncomputed, by ensuring these sub-expressions are **lifted**.

Format of Typing Rules. In Fig. 7, $\Gamma \vdash^\alpha e : \tau$ indicates an expression e has type τ under context Γ , and the evaluation of e is $\alpha \subseteq \{\mathbf{qfree}, \mathbf{mfree}\}$. For example, $x : \mathbb{B} \vdash^\alpha H(x) : \mathbb{B}$ for $\alpha = \{\mathbf{mfree}\}$, where **mfree** $\in \alpha$ since evaluating $H(x)$ does not induce a measurement, and **qfree** $\notin \alpha$ since the effect of evaluating $H(x)$ cannot be described classically. We note that in general, $x : \tau \vdash^\alpha f(x) : \tau'$ if f has type $\tau' \xrightarrow{\alpha} \tau'$, i.e., the annotation of f determines the annotation of the turnstile $\vdash$.

A context Γ is a multiset $\{\beta_i x_i : \tau_i\}_{i \in I}$ that assigns a type τ_i to each variable x_i , where I is a finite index set, and x_i may be annotated by **const** $\in \beta_i$, indicating that it will not be consumed during evaluation of e . As a shorthand, we often write $\Gamma = \vec{\beta}\vec{x} : \vec{\tau}$.

We write $\Gamma, \beta x : \tau$ for $\Gamma \uplus \{\beta x : \tau\}$, where $\uplus$ denotes the union of multisets. Analogously Γ, Γ' denotes $\Gamma \uplus \Gamma'$. In general, we require that types and annotations of contexts can never be conflicting, i.e., $\beta x : \tau \in \Gamma$ and $\beta' x : \tau' \in \Gamma$ implies $\beta = \beta'$ and $\tau = \tau'$.

6.1 Typing Constants and Variables

If c is a constant of type τ , its typing judgement is given by $\emptyset \vdash^{\mathbf{mfree}, \mathbf{qfree}} c : \tau$. For example, $\emptyset \vdash^{\mathbf{mfree}, \mathbf{qfree}} H : \mathbb{B} \xrightarrow{\mathbf{mfree}} \mathbb{B}$.

Here, we annotate the turnstile $\vdash$ as **qfree**, because evaluating expression H maps the empty state $| \rangle$ to $| \rangle \otimes | H \rangle_H$, which can be described classically by $\overline{f}(| \rangle) = | H \rangle_H$. We provide the types of other selected built-in functions in App. E.2.

Likewise, the typing judgement of variables carries annotations **qfree** and **mfree** (rule var in Fig. 8), as all constants c and variables x in Silq-core can be evaluated without measurement, and their semantics can be described classically. Further, both rules assume an empty context (for constants c) or a context consisting only of the evaluated variable (for variables), preventing ignoring variables from the context. To drop constant and classical variables from the context, we introduce an explicit weakening rule, discussed next.

Weakening and Contraction. Fig. 8 further shows weakening and contraction typing rules for classical and constant variables. These rules allow us to drop classical and constant variables from the context (weakening rules $!W$ and W) and duplicate them (contraction rules $!C$ and C). For weakening, the interpretation of “dropping variable x ” arises from reading the rule bottom-up, which is also the way our semantics operates (analogously for contraction). In our semantics (§7), dropping constant variable x in the body of a function f can be handled by uncomputing it at the end of f .

$$\begin{array}{c}
\frac{}{\beta x : \tau \vdash^{\text{mfree}, \text{qfree}} x : \tau} \text{var} \quad \frac{\Gamma \vdash^{\alpha} e : \tau'}{\Gamma, x : !\tau \vdash^{\alpha} e : \tau'} !W \quad \frac{\Gamma \vdash^{\alpha} e : \tau'}{\Gamma, \text{const } x : \tau \vdash^{\alpha} e : \tau'} W \quad \frac{\Gamma, x : !\tau, x : !\tau \vdash^{\alpha} e : \tau'}{\Gamma, x : !\tau \vdash^{\alpha} e : \tau'} !C \quad \frac{\Gamma, \text{const } x : \tau, \text{const } x : \tau \vdash^{\alpha} e : \tau'}{\Gamma, \text{const } x : \tau \vdash^{\alpha} e : \tau'} C
\end{array}$$

Figure 8. Typing variables, including weakening and contraction.

$$\frac{\Gamma_i \vdash^{\alpha_i} e_i : \tau_i \quad \Gamma' \vdash^{\alpha'} e' : \prod_{i=1}^n \beta_i \tau_i ! \xrightarrow{\alpha'_{\text{func}}} \tau'}{\Gamma_1, \dots, \Gamma_n, \Gamma' \vdash^{\alpha''} e'(e_1, \dots, e_n) : \tau'} \text{func-eval}$$

$$\text{const} \in \beta_i \implies \text{qfree} \in \alpha_i \wedge \Gamma_i = \text{const } \vec{x} : \vec{\tau}'' \quad (1)$$

$$\text{qfree} \in \alpha'' \iff \text{qfree} \in \bigcap_i \alpha_i \cap \alpha' \cap \alpha'_{\text{func}} \quad (2)$$

$$\text{mfree} \in \alpha'' \iff \text{mfree} \in \bigcap_i \alpha_i \cap \alpha' \cap \alpha'_{\text{func}} \quad (3)$$

Figure 9. Typing rule and constraints for function calls.

We note that variables with classical type can be used more liberally than **const** variables (e.g., as if-conditions). Hence, annotating a classical variable as **const** has no effect. We annotate variables (not types) as **const** as our syntax does not allow partially consuming variables.

6.2 Measurement

We type **measure** as $\emptyset \vdash^{\text{mfree}, \text{qfree}} \text{measure} : \tau \rightarrow !\tau$, where the lack of **const** annotation for τ indicates **measure** consumes its argument, and $!\tau$ indicates the result is classical. We annotate the judgement itself as **mfree**, as evaluating the expression **measure** simply yields the function **measure**, without inducing a measurement. In contrast, the function type of **measure** itself is not **mfree** (indicated by $! \rightarrow$), as evaluating the function **measure** on an argument induces a measurement. Thus, **measure**(0) is not **mfree**, as evaluating it induces a measurement: $\emptyset \vdash \text{measure}(0) : !\mathbb{B}$.

6.3 Function Calls

Fig. 9 shows the typing rule for function calls $e'(e_1, \dots, e_n)$. Ignoring annotations, the rule follows the standard pattern, which we provide for convenience in App. E.1 (Fig. 25).

We now discuss the annotation Constraints (1)–(3). Constraint (1) ensures that if a function leaves its argument constant ($\text{const} \in \beta_i$), e_i is **lifted**, i.e., **qfree** and depending only on **const** variables. In turn, this ensures that we can automatically uncompute e_i when it is no longer needed. We note that non-constant arguments ($\text{const} \notin \beta_i$) do not need to be uncomputed, as they are no longer available after the call. To illustrate that Constraint (1) is critical, consider a function $f : \text{const } \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$, and a call $f(x + 1, \mathbf{H}(x))$ with non-constant variable x in context Γ_1 . This call must be rejected by Silq-core as there is no way to uncompute $x + 1$ after the call to f , since $\mathbf{H}$ consumes x . Indeed, since x is not **const** in Γ_1 even though $\text{const} \in \beta_1$, (1) does not hold.

$$\frac{\vec{\beta}\vec{x} : \vec{\tau}, \vec{\beta}'\vec{y} : !\vec{\tau}' \vdash^{\alpha} e : \tau''}{\vec{\beta}'\vec{y} : !\vec{\tau}' \vdash^{\text{mfree}, \text{qfree}} \lambda(\vec{\beta}\vec{x} : \vec{\tau}).e : \prod_{i=1}^n \beta_i \tau_i ! \xrightarrow{\alpha} \tau''} \lambda\text{-abs}$$

Figure 10. Typing lambda abstraction.

Constraint (2) ensures an expression is only **qfree** if all its components are **qfree**, and if the evaluated function only uses **qfree** operations. Constraint (3) is analogous for **mfree**.

We note that in order to allow temporarily marking variables as **const** (as discussed in §4.4), we could replace the top-left Γ_i in Fig. 9 by $\Gamma_i, \star_i$, where $\star_i = \text{const } \Gamma_{i+1}, \dots, \text{const } \Gamma_n$ if $\text{const} \notin \beta_i$, and $\star_i = \emptyset$ otherwise. This would allow us to temporarily treat variables as **const**, if they appear in a consumed expression e_i and they are consumed in a later expression e_j for $j > i$. Fig. 9 omits this for conciseness.

6.4 Lambda Abstraction

Fig. 10 shows the rule for lambda abstraction. Its basic pattern without annotations is again standard (App. E.1). In terms of annotations, the rule enforces multiple constraints. First, it ensures that the annotation of the abstracted function follows the annotation α of the original typing judgment. Second, we tag the resulting type judgment as **mfree** and **qfree**, since function abstraction requires neither measurement nor quantum operations. Third, the rule allows capturing classical variables (y_i has type $!\tau_i$), but not quantum variables. This ensures that all functions in Silq-core are classically known, i.e., can be described by a classical state.

6.5 Reverse

Fig. 11 shows the type of **reverse**. We only allow reversing functions without classical components in input or output types (indicated by $\mathcal{X}$), as reconstructing classical components of inputs is typically impossible. Concretely, types without classical components are (i) $\mathbb{1}$, (ii) $\mathbb{B}$, and (iii) products of types without classical components. In particular, this rules out all classical types $!\tau$, function types, and products of types with classical components.

The input to **reverse**, i.e., the function f to be reversed must be measure-free, because measurement is irreversible. Further, the function f may or may not be **qfree** (as indicated by a callout). Then, the type rule for **reverse** splits the input types of f into constant and non-constant ones. The depicted rule assumes the first parameters of f are annotated as constant, but we can easily generalize this rule to other

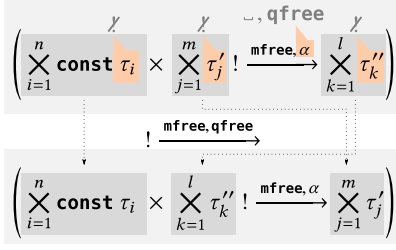
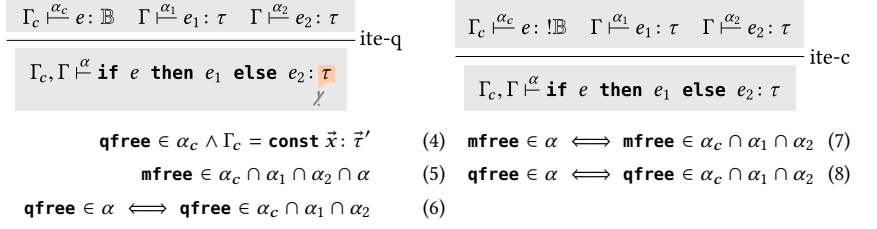
Figure 11. Type of **reverse**.

Figure 12. Typing quantum (left) and classical (right) control flow

Classical set $\llbracket \tau \rrbracket^c$			
$\llbracket \mathbb{1} \rrbracket^c := \{\emptyset\}$	$\llbracket \mathbb{B} \rrbracket^c := \{\emptyset\}$	$\llbracket \prod_{k=1}^n \tau_k \rrbracket^c := \prod_{k=1}^n \llbracket \tau_k \rrbracket^c$	$\llbracket \prod_{k=1}^n \beta_k \tau_k ! \xrightarrow{\alpha} \tau' \rrbracket^c := \left\{ e, \sigma \mid \begin{array}{l} \vec{y} : !\vec{\tau}'' \vdash^{\alpha'} e : \prod_{k=1}^n \beta_k \tau_k ! \xrightarrow{\alpha} \tau', \\ \sigma \in \llbracket \vec{y} : !\vec{\tau}'' \rrbracket^c \end{array} \right\}$
Quantum ground set $\llbracket \tau \rrbracket^q$			
$\llbracket \mathbb{1} \rrbracket^q := \{\emptyset\}$	$\llbracket \mathbb{B} \rrbracket^q := \{\emptyset, 1\}$	$\llbracket \prod_{k=1}^n \tau_k \rrbracket^q := \prod_{k=1}^n \llbracket \tau_k \rrbracket^q$	$\llbracket \prod_{k=1}^n \beta_k \tau_k ! \xrightarrow{\alpha} \tau' \rrbracket^q := \{\emptyset\}$
			$\llbracket !\tau \rrbracket^q := \{\emptyset\}$

Figure 13. Classical set $\llbracket \tau \rrbracket^c$ and quantum ground set $\llbracket \tau \rrbracket^q$ to build the semantics $\llbracket \tau \rrbracket = \llbracket \tau \rrbracket^c \times \mathcal{H}(\llbracket \tau \rrbracket^q)$ of type τ .

orders. Based on this separation, **reverse** returns a function which starts from the constant input types and the output types of f , and returns the non-constant input types. The returned function **reverse**(f) is measure-free, and **qfree** if f is **qfree**.

6.6 Control Flow

Even though **if** e **then** e_1 **else** e_2 is syntactically standard, it supports both classical and quantum conditions e . A classical condition induces classical control flow, while a non-classical (i.e., quantum) condition induces quantum control flow. In Fig. 12, we provide the typing rules for both cases, which follow the standard basic patterns when ignoring annotations (App. E.1).

Quantum Control Flow. Constraint (4) ensures that e is **lifted** and can thus be uncomputed after the conditional, analogously to uncomputing constant arguments in Constraint (1). Constraint (5) requires both branches to be **mfree**, which is important because we cannot condition a measurement on a quantum value (this would violate physicality). Further, it also requires the condition to be **mfree** (which is already implicitly ensured by Constraint (4) as all **qfree** expressions are also **mfree**), meaning the whole expression is **mfree**. Constraint (6) ensures that the resulting typing judgment gets tagged as **qfree** if all subexpressions are **qfree**. Finally, the rule does not allow the return type τ to contain classical components (indicated by γ), as otherwise we could introduce unexpected superpositions of classical values.

Classical Control Flow. Classical control flow requires the condition to be classical, in addition to our usual restrictions on annotations. Concretely, Constraints (7) and (8) propagate **mfree** and **qfree** annotations.

7 Semantics of Silq-Core

In this section, we discuss the operational semantics of Silq-Core. We use big-step semantics, as this is more convenient to define reverse and control flow.

7.1 Semantics of Types

We build the semantics $\llbracket \tau \rrbracket$ of type τ from a *classical set* $\llbracket \tau \rrbracket^c$ and a *quantum ground set* $\llbracket \tau \rrbracket^q$ as $\llbracket \tau \rrbracket = \llbracket \tau \rrbracket^c \times \mathcal{H}(\llbracket \tau \rrbracket^q)$. Note that $\llbracket \tau \rrbracket$ stores the classical and quantum parts of τ separately, which is in line with how a QRAM can physically store values of type τ . In particular, $\llbracket \tau \rrbracket^q$ contains the ground set from which we build the Hilbert space $\mathcal{H}(\llbracket \tau \rrbracket^q)$.

Classical Set and Quantum Ground Set. Fig. 13 defines both the classical set $\llbracket \tau \rrbracket^c$ and the quantum ground set $\llbracket \tau \rrbracket^q$ for all possible types τ . For type $\mathbb{1}$, both the classical set and the quantum ground set are the singleton set $\{\emptyset\}$. The (quantum) Boolean type $\mathbb{B}$ stores no classical information and hence, its classical set is again the singleton set. In contrast, its quantum ground set is $\{\emptyset, 1\}$, for which $\mathcal{H}(\{\emptyset, 1\})$ contains all superpositions of $|0\rangle$ and $|1\rangle$. The sets associated with the product type are standard. Functions store no quantum information, and hence their quantum ground set is $\{\emptyset\}$. In contrast, the classical set associated with a function type contains all expressions e of this type, and a state σ storing the variables captured in e . Finally, classical types $!\tau$ store no quantum information and hence their quantum ground set is $\{\emptyset\}$. In contrast, their classical set consists of (i) the classical set $\llbracket \tau \rrbracket^c$ which remains classical and (ii) the quantum ground set $\llbracket \tau \rrbracket^q$. As a straightforward consequence of our definition, duplicate classical annotations do not affect the semantics: $\llbracket !!\tau \rrbracket \simeq \llbracket !\tau \rrbracket$.

Type semantics	$\begin{aligned} \llbracket !\mathbb{B} \rrbracket &= \llbracket !\mathbb{B} \rrbracket^c \times \mathcal{H}(\llbracket !\mathbb{B} \rrbracket^q) = (\llbracket \mathbb{B} \rrbracket^c \times \llbracket \mathbb{B} \rrbracket^q) \times \mathcal{H}(\{()\}) = (\{()\} \times \{0, 1\}) \times \mathcal{H}(\{()\}) \simeq \{0, 1\} \times \mathcal{H}(\{()\}) \\ \llbracket !\mathbb{B} \times \mathbb{B} \rrbracket &= \llbracket !\mathbb{B} \times \mathbb{B} \rrbracket^c \times \mathcal{H}(\llbracket !\mathbb{B} \times \mathbb{B} \rrbracket^q) = (\llbracket !\mathbb{B} \rrbracket^c \times \llbracket \mathbb{B} \rrbracket^c) \times \mathcal{H}(\llbracket !\mathbb{B} \rrbracket^q \times \llbracket \mathbb{B} \rrbracket^q) = (\{()\} \times \{0, 1\} \times \{()\}) \times \mathcal{H}(\{()\} \times \{0, 1\}) \simeq \{0, 1\} \times \mathcal{H}(\{0, 1\}) \\ \llbracket !\mathbb{B} \times \mathbb{B} \rrbracket^+ &= \mathcal{H}(\llbracket !\mathbb{B} \rrbracket^c \times \llbracket \mathbb{B} \rrbracket^q) \simeq \mathcal{H}(\{0, 1\} \times \{0, 1\}) = \mathcal{H}(\{0, 1\}^2) = \left\{ \sum_{w \in \{0, 1\}^2} \gamma_w w\rangle \mid \gamma_w \in \mathbb{C} \right\} \end{aligned}$
Context semantics	$\begin{aligned} \llbracket x : !\mathbb{B} \times \mathbb{B} \rrbracket &= \llbracket x : !\mathbb{B} \times \mathbb{B} \rrbracket^c \times \mathcal{H}(\llbracket x : !\mathbb{B} \times \mathbb{B} \rrbracket^q) \simeq \{(v)_x \mid v \in \{0, 1\}\} \times \mathcal{H}(\{(v')_x \mid v' \in \{0, 1\}\}) = \left\{ \left((v)_x, \gamma_0 0\rangle_x + \gamma_1 1\rangle_x \right) \mid \begin{array}{l} v \in \{0, 1\} \\ \gamma_0, \gamma_1 \in \mathbb{C} \end{array} \right\} \\ \llbracket x : !\mathbb{B} \times \mathbb{B} \rrbracket^+ &= \mathcal{H}(\llbracket x : !\mathbb{B} \times \mathbb{B} \rrbracket^c \times \llbracket x : !\mathbb{B} \times \mathbb{B} \rrbracket^q) \simeq \mathcal{H}\left(\left\{ \underbrace{((v)_x, (v')_x)}_{(v, v')_x} \mid \begin{array}{l} v \in \{0, 1\} \\ v' \in \{0, 1\} \end{array} \right\}\right) = \left\{ \sum_{w \in \{0, 1\}^2} \gamma_w w\rangle_x \mid \gamma_w \in \mathbb{C} \right\} \end{aligned}$
Embedding	$\iota(\llbracket x : !\mathbb{B} \times \mathbb{B} \rrbracket) \simeq \left\{ \iota((v)_x, \gamma_0 0\rangle_x + \gamma_1 1\rangle_x) \mid v \in \{0, 1\}, \gamma_0, \gamma_1 \in \mathbb{C} \right\} \simeq \left\{ \gamma_0 v, 0\rangle_x + \gamma_1 v, 1\rangle_x \mid v \in \{0, 1\}, \gamma_0, \gamma_1 \in \mathbb{C} \right\}$

Figure 14. Example semantics of type $!\mathbb{B}$, type $!\mathbb{B} \times \mathbb{B}$, and context $x : !\mathbb{B} \times \mathbb{B}$.

$$\begin{aligned} \llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket &:= \llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^c \times \mathcal{H}(\llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^q) = \{(v_1)_{x_1}, \dots, (v_n)_{x_n} \mid v_i \in \llbracket \tau_i \rrbracket^c\} \times \mathcal{H}(\{(v'_1)_{x_1}, \dots, (v'_n)_{x_n} \mid v'_i \in \llbracket \tau_i \rrbracket^q\}) \\ \llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^+ &:= \mathcal{H}(\llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^c \times \llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^q) \simeq \left\{ \sum_{w_i \in \llbracket \tau_i \rrbracket^c \times \llbracket \tau_i \rrbracket^q} \gamma_{w_1, \dots, w_n} |w_1\rangle_{x_1} \otimes \dots \otimes |w_n\rangle_{x_n} \mid \gamma_{w_1, \dots, w_n} \in \mathbb{C} \right\} \end{aligned}$$

Figure 15. Semantics $\llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket$ and extended semantics $\llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^+$ of context $\vec{\beta}\vec{x} : \vec{\tau}$.

To illustrate the semantics of types, Fig. 14 provides semantics for two example types. In particular, $!\mathbb{B}$ is isomorphic to $\{0, 1\} \times \mathcal{H}(\{()\})$ – note that it is not formally isomorphic to $\{0, 1\}$ because $\mathcal{H}(\{()\}) = \{\gamma |()\rangle \mid \gamma \in \mathbb{C}\}$ tracks a (physically irrelevant) global phase $\gamma \in \mathbb{C}$.

Extended Semantic Space. Unfortunately, working with elements $(v, \varphi) \in \llbracket \tau \rrbracket$ for $v \in \llbracket \tau \rrbracket^c$ and $\varphi \in \mathcal{H}(\llbracket \tau \rrbracket^q)$ is inconvenient because (i) every operation on (v, φ) needs to handle v and φ separately (as they are different mathematical objects) and (ii) some operations, like $(v, \varphi) + (v', \varphi')$ are not defined because $\llbracket \tau \rrbracket$ is not a vector space.

Therefore, we define the semantics of expressions (§7.2) and annotations (§7.4) on a more convenient, larger space that also allows superpositions of classical values:

$$\llbracket \tau \rrbracket^+ := \mathcal{H}(\llbracket \tau \rrbracket^c \times \llbracket \tau \rrbracket^q) = \mathcal{H}(\llbracket \tau \rrbracket^c) \otimes \mathcal{H}(\llbracket \tau \rrbracket^q).$$

Here, (i) the classical and quantum part of $\phi \in \llbracket \tau \rrbracket^+$ can be handled analogously and (ii) operation $+$ is defined on $\llbracket \tau \rrbracket^+$. We provide an example of this extended semantics in Fig. 14.

While elements of $\llbracket \tau \rrbracket$ can be naturally lifted to $\llbracket \tau \rrbracket^+$ via embedding $\iota : \llbracket \tau \rrbracket \rightarrow \llbracket \tau \rrbracket^+$ defined by $\iota(v, \varphi) := |v\rangle \otimes \varphi$, the larger space $\llbracket \tau \rrbracket^+$ also contains *invalid elements*, namely those where classical values are in superposition. In contrast, *valid elements* do not put classical values in superposition, i.e., the classical part of every summand in their superposition coincides. Our semantics exclusively produces valid elements, as we formally prove in Thm. 7.1.

Semantics of Contexts. Fig. 15 provides the semantics of context $\llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket$. Here, $(v_i)_{x_i}$ indicates that variable x_i stores value v_i . Fig. 14 provides semantics for an example context, where we write $|0\rangle_x$ as a short-hand for $|()\rangle_x$.

Analogously to $\llbracket \tau \rrbracket^+$, Fig. 15 also introduces the extended semantics $\llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^+$ for contexts, and a standard representation that stores the classical and quantum value of variable x together in a single location $|v\rangle_x$. We use this representation throughout this work (including Fig. 3). Again, we illustrate this extended semantics in Fig. 14.

For contexts, the embedding $\iota : \llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket \rightarrow \llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^+$ is

$$\iota((\vec{v})_{\vec{x}}, \sum_{\vec{v}'} \gamma_{\vec{v}'} |\vec{v}'\rangle) = \sum_{\vec{v}'} \gamma_{\vec{v}'} |\vec{v}, \vec{v}'\rangle_{\vec{x}},$$

for $(\vec{v})_{\vec{x}} \in \llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^c$ and $\sum_{\vec{v}'} \gamma_{\vec{v}'} |\vec{v}'\rangle_{\vec{x}} \in \mathcal{H}(\llbracket \vec{\beta}\vec{x} : \vec{\tau} \rrbracket^q)$. We illustrate this in Fig. 14 on an example context.

7.2 Semantics of Expressions

Our operational semantics evaluates an expression e in state ψ by constructing derivation trees whose structure follows the structure of our type derivations. Since e may contain measurements with probabilistic outcome, we provide an evaluation $\llbracket \Gamma \vdash^\alpha e : \tau \mid \psi \rrbracket \xrightarrow{\text{run}} \psi'_i$ for each possible sequence of measurement results, indicating that evaluating e (typed as $\Gamma \vdash^\alpha e : \tau$), on state ψ yields state ψ'_i with probability $\|\psi'_i\|^2$,

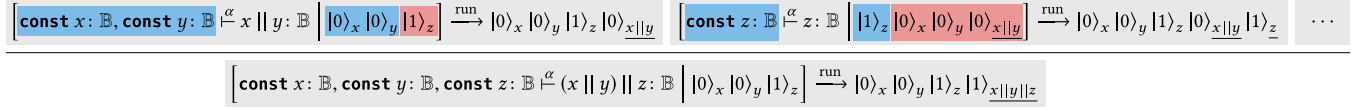


Figure 16. Part of derivation when evaluating expression $x \parallel y \parallel z$ on state $\psi = |0\rangle_x |0\rangle_y |1\rangle_z$. For a complete semantics derivation tree that leverages more rules, see App. F.3.

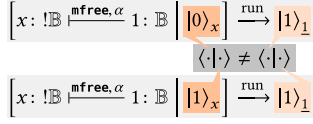


Figure 17. Non-isometry.

assuming $\|\psi\|^2 = 1$ (see §3). If e is undefined for a given input ψ (possible since **reverse** returns unsafe functions), we do not provide any evaluation.

Domain of ψ, ψ' . When our semantics evaluates e according to $\left[\Gamma \vdash e : \tau'' \mid \psi \right] \xrightarrow{\text{run}} \psi'$, it requires that $\psi \in \iota(\llbracket \Gamma, \Delta \rrbracket)$. By construction, for context $\Gamma = \text{const } \vec{x} : \vec{\tau}, \vec{y} : \vec{\tau}'$, output state ψ' lies in $\llbracket \text{const } \vec{x} : \vec{\tau}, \Delta, \underline{e} : \tau'' \rrbracket^+$, i.e., we preserve constant variables $\vec{x}$ and the additional context Δ (discussed next), and store the value of e in a temporary variable $\underline{e}$.

Here, Δ is additional context containing the remainder of the state preserved while evaluating e . We illustrate the need for Δ in Fig. 16. Here, we cannot evaluate $(x \parallel y)$ and z independently, as their values may be entangled in ψ . Hence, we must evaluate $x \parallel y$ in a state that not only contains x, y (in the context when typing $x \parallel y$, cp. blue box), but also z (in Δ , cp. red box). After this, we evaluate z in a state containing z (in the context when typing z), x, y (in Δ), and the value of $x \parallel y$ (stored as $x \parallel y$ in Δ).

Formal Semantics. Here, we discuss the most important aspects of the formal semantics of Silq-core expressions (see App. F.1 for details, and App. F.3 for an example). Recall that the structure of semantic derivation trees follows the structure of the type derivation trees, and hence, every type rule corresponds to a semantic derivation rule.

The semantics of evaluating a variable x is to rename x to $\underline{x}$ in the new state if x is consumed, and to duplicate x to $\underline{x}$ if x is constant. Contraction of constant variable x duplicates x , according to $\sum_v \gamma_v |v\rangle_x \otimes \tilde{\psi}_v \mapsto \sum_v \gamma_v |v\rangle_x |v\rangle_x \otimes \tilde{\psi}_v$. Weakening of constant variables postpones uncomputing them until the end of the function body. When evaluating a function call $e'(e_1, \dots, e_n)$, we uncompute the constant arguments e_i (i.e., preserved according to the signature of e') at the end of the function call. To reverse functions, we postpone the reversal until the reversed function is called. We handle control flow **if** e **then** e_1 **else** e_2 by separately evaluating e_1 (respectively e_2) in the part of the state where e is true (respectively false).

7.3 Type Preservation

Thm. 7.1 ensures that our semantics never produces invalid states ψ' , meaning that the classical values in ψ' can never be in superposition (since ι only returns *valid* elements).

Theorem 7.1 (Type Preservation). *If $\Gamma = \text{const } \vec{x} : \vec{\tau}, \vec{y} : \vec{\tau}'$, $\left[\Gamma \vdash e : \tau'' \mid \psi \right] \xrightarrow{\text{run}} \psi'$, and $\psi \in \iota(\llbracket \Gamma, \Delta \rrbracket)$, then ψ' lies in $\iota(\llbracket \text{const } \vec{x} : \vec{\tau}, \underline{e} : \tau'', \Delta \rrbracket)$.*

We provide a proof for Thm. 7.1 in App. G. Here, $\iota(\llbracket \Gamma, \Delta \rrbracket)$ contains all elements of $\llbracket \Gamma, \Delta \rrbracket^+$ where classical values are not in superposition.

7.4 Semantics of Annotations

In the following, we show theorems formalizing the guarantees of annotations of Silq-core expressions. We do not formally discuss the guarantees of annotations of Silq-core functions, which are analogous. We note that the guarantees of $!$ were already discussed in §7.3.

Preserving Constants. Thm. 7.2 ensures that constant variables are indeed preserved by Silq-core.

Theorem 7.2 (Const Semantics). *If $\Gamma = \text{const } \vec{x} : \vec{\tau}, \vec{y} : \vec{\tau}'$, $\left[\Gamma \vdash e : \tau'' \mid \psi \right] \xrightarrow{\text{run}} \psi'$, and $\psi = \sum_{\vec{v}, \vec{w}} \gamma_{\vec{v}, \vec{w}} |\vec{v}\rangle_{\vec{x}} \otimes |\vec{w}\rangle_{\vec{y}} \otimes \tilde{\psi}_{\vec{v}, \vec{w}}$, then $\psi' = \sum_{\vec{v}, \vec{w}} \gamma_{\vec{v}, \vec{w}} |\vec{v}\rangle_{\vec{x}} \otimes \chi_{\vec{v}, \vec{w}} \otimes \tilde{\psi}_{\vec{v}, \vec{w}}$ for some $\chi_{\vec{v}, \vec{w}}$.*

We provide a proof for Thm. 7.2 in App. G.

Mfree Expressions. We want to ensure that **mfree** expressions correspond to linear isometries, which in turn ensures we can physically implement their effect with quantum gates. However, this correspondence is non-trivial: Fig. 17 shows an example where $\xrightarrow{\text{run}}$ is not isometric because we drop a classical value from the input. Thm. 7.3 side-steps this issue, intuitively by ensuring that our semantics is isometric when the classical components of its input are fixed.

Theorem 7.3 (Mfree Semantics). *If $\text{mfree} \in \alpha$, $\sigma \in \llbracket \Gamma, \Delta \rrbracket^c$,*

$$\left[\Gamma \vdash e : \tau'' \mid \iota(\sigma, \psi_1) \right] \xrightarrow{\text{run}} \psi'_1 \quad \text{for } \psi_1 \in \mathcal{H}(\llbracket \Gamma, \Delta \rrbracket^q), \text{ and}$$

$$\left[\Gamma \vdash e : \tau'' \mid \iota(\sigma, \psi_2) \right] \xrightarrow{\text{run}} \psi'_2 \quad \text{for } \psi_2 \in \mathcal{H}(\llbracket \Gamma, \Delta \rrbracket^q),$$

$$\text{then } \langle \psi_1 | \psi_2 \rangle = \langle \psi'_1 | \psi'_2 \rangle.$$

We provide a proof for Thm. 7.3 in App. G.

A useful interpretation of Thm. 7.3 states that $\xrightarrow{\text{run}}$ acts like an isometry on the subspace consistent with a fixed classical component $\sigma \in \llbracket \Gamma, \Delta \rrbracket^c$,

$$\{\iota(\sigma, \chi) \mid \chi \in \mathcal{H}(\llbracket \Gamma, \Delta \rrbracket^q)\} \subseteq \llbracket \Gamma, \Delta \rrbracket^+.$$

This corresponds to the intuition that in order to evaluate e on ψ_1 , we can (i) extract the classical component σ from ψ_1 , (ii) build a circuit C that realizes the linear isometry for this classical component and (iii) run C , yielding ψ'_1 .

Qfree Expressions. Thm. 7.4 ensures that qfree expressions can be described by a function $\tilde{f}$ on the ground sets.

Theorem 7.4 (Qfree Semantics). *If $\Gamma \vdash^\alpha e : \tau''$ for **qfree** $\in \alpha$ and context $\Gamma = \text{const } \vec{x} : \vec{\tau}, \vec{y} : \vec{\tau}'$, then there exists a function $\tilde{f} : \llbracket \Gamma \rrbracket^s \rightarrow \llbracket \text{const } \vec{x} : \vec{\tau}, \vec{y} : \vec{\tau}' \rrbracket^s$ on ground sets such that*

$$\left[\Gamma \vdash^\alpha e : \tau'' \mid \sum_{\sigma \in \llbracket \Gamma \rrbracket^s} \gamma_\sigma |\sigma\rangle \otimes \tilde{\psi}_\sigma \right] \xrightarrow{\text{run}} \sum_{\sigma \in \llbracket \Gamma \rrbracket^s} \gamma_\sigma |\tilde{f}(\sigma)\rangle \otimes \tilde{\psi}_\sigma,$$

where $\llbracket \Gamma \rrbracket^s$ is a shorthand for the ground set $\llbracket \Gamma \rrbracket^c \times \llbracket \Gamma \rrbracket^q$ on which the Hilbert space $\llbracket \Gamma \rrbracket^+ = \mathcal{H}(\llbracket \Gamma \rrbracket^s)$ is defined.

We provide a proof for Thm. 7.4 in App. G.

7.5 Physicality

Thm. 7.5 ensures Silq-core programs can be physically realized on a QRAM. If we would change our semantics to abort on operations that are not physical, we could re-interpret Thm. 7.5 to guarantee *progress*, i.e., the absence of errors due to unphysical operations.

Theorem 7.5 (Physicality). *The semantics of well-typed Silq programs is physically realizable on a QRAM.*

We provide a proof for Thm. 7.5 in App. G, which heavily relies on the semantics of annotations. As a key part of the proof, we show that we can uncompute temporary values by reversing the computation that computed them. Reversing a computation is possible on a QRAM (and supported by most existing quantum languages) by (i) producing the gates that perform this computation and (ii) reversing them.

8 Evaluation of Silq

Next, we experimentally compare Silq to other languages. Our comparison focuses on Q#, because (i) it is one of the most widely used quantum programming languages, (ii) we consider it to be more high-level than Cirq or QisKit, and (iii) the Q# coding contest [15, 16] provides a large collection of Q# implementations we can leverage for our comparison. To check if our findings can generalize to other languages, we also compare Silq to Quipper (§8.2).

Table 1. Silq compared to Q#.

	Silq			Q#		
	S18	W19	Both	S18	W19	Both
Lines of code	99	168	267	251	242	493
Quantum primitives	8	10	10	12	19	22
Annotations	2	3	3	3	6	6
Low-level quantum gates	14	23	37	33	54	87

Implementation. We implemented a publicly available parser, type-checker, and simulator for Silq as a fork of the PSI probabilistic programming language [6]. Specifically, Silq’s AST and type checker are based on PSI, while Silq’s simulator is independent of PSI. Our implementation handles all valid Silq code examples in this paper, while rejecting invalid programs. We also provide a development environment for Silq, in the form of a Visual Studio Code extension.¹⁰

Compared to Silq-core, Silq supports an imperative fragment (including automatic uncomputation), additional primitives, additional convenience features (e.g., unpacking of tuples), additional types (e.g., arrays), dependent types (which only depend on classical values, as shown in Fig. 3), type equivalences (e.g., $!!\tau \equiv !\tau$), subtyping, and type conversions.

8.1 Comparing Silq to Q#

To compare Silq to Q#, we solved all 28 tasks of Microsoft’s Q# Summer 2018 and Winter 2019 [15, 16] coding contest in Silq. We compared the Silq solutions to the Q# reference solutions provided by the language designers [17, 18] (Tab. 1) and the top 10 contestants (App. H).

Our results indicate that algorithms expressed in Silq are far more concise compared to the reference solution (−46%) and the average top 10 contestants (−59%). We stress that we specifically selected these baselines to be written by experts in Q# (for reference solutions) or strong programmers well-versed in Q# (for top 10 contestants). We did not count empty lines, comments, import statements, namespace statements, or lines that were unreachable for the method solving the task. This greatly benefits Q#, as it requires various imports.

Because the number of lines of code heavily depends on the available language features, we also counted (i) the number of different quantum primitives, (ii) the number of different annotations in both Q# (controlled auto, adjoint self, Controlled, ...) and Silq (**mfree**, **qfree**, **const**, **lifted**, and **!**), as well as (iii) the number of low-level quantum circuit gates used to encode all programs in Q# and Silq (for details, see App. H).

Our results demonstrate that Silq is not only significantly more concise, but also requires only half as many quantum primitives, annotations, and low-level quantum gates compared to Q#. As a consequence, we believe Silq programs are easier to read and write. In fact, we conjecture that the

¹⁰<https://marketplace.visualstudio.com/items?itemName=eth-sri.vscode-silq>

Table 2. Comparing Silq to other quantum languages. Parenthesized features are partially (but not fully) supported.

Language	Type system	Autom. Uncomp.	const	mfree	qfree
QPL [27]	linear ¹¹	✗	✗	✗	✗
Quantum λ -calc. [28]	affine	✗	✗	✗	✗
Quipper [7]	non-linear	(✓)	✗	✗	✗
ReVerC [2]	non-linear	(✓)	✗	✗	(✗)
QWire [22]	linear	✗	✗	✗	✗
Q# [30]	non-linear	✗	✗	✓	✗
ReQWire [23]	linear	(✗)	(✗)	✗	(✗)
Silq (this work)	linear+	✓	✓	✓	✓

code of the top 10 contestants was longer than the reference solutions because they had difficulties choosing the right tools out of Q#’s large set of quantum primitives. We note that Silq is better in abstracting away standard low-level quantum circuit gates: they occur only half as often in Silq.

8.2 Comparing Silq to Quipper

The language designers of Quipper provide an encoding [26] of the triangle finding algorithm [4, 14]. We encoded this algorithm in Silq and found that again, we need significantly less code (−38%; Quipper: 378 LOC, Silq: 236 LOC). An excerpt of this, on which we achieve even greater reduction (−64%), was already discussed in Fig. 2.

The intent of the algorithm in Fig. 2 is naturally captured in Silq: it iterates over all j, k with $0 \leq j < k < 2^{\text{rbar}}$, and counts how often `ee[tau[j]][tau[k]] && eew[j] && eew[k]` is true, where we use quantum indexing into `ee`. In contrast, Quipper’s code is cluttered by explicit uncomputation (e.g., of `eedd_k`), custom functions aiding uncomputation (e.g., `.&&.`), and separate initialization and assignment (e.g., `eedd_k`), because Quipper lacks automatic uncomputation.

Similarly to Q#, Quipper offers an abundance of built-in and library functions. It supports 76 basic gates and 8 types of reverse, while Silq only provides 10 basic gates and 1 type of reverse, without sacrificing expressivity. Some of Quippers overhead is due to double definitions for programming in declarative and imperative style, e.g., it offers both `gate_T` and `gate_T_at` or due to definition of inverse gates, e.g., `gate_T_inv`.

8.3 Further Silq Implementations

To further illustrate the expressiveness of Silq on interesting quantum algorithms, we provide Silq implementations of (i) Wiesner’s quantum money scheme [34], (ii) a naive (unsuccessful) attack on it, and (iii) a recent (successful) attack on it [20] in App. H.3.

¹¹QPL (i) enforces no-cloning syntactically and (ii) disallows implicitly dropping variables (cp. rule `discard` in Fig. 12)

9 Related Work

Various quantum programming languages aim to simplify development of quantum algorithms. Tab. 2 shows the key language features of the languages most related to ours.

Const. To our knowledge, Silq is the first quantum language to mark variables as constant. We note that for Q#, so-called *immutable* variables can still be modified (unlike **const** variables), for example by applying the Hadamard transform **H**.

Silq’s constant annotation is related to ownership type systems guaranteeing read-only references [19]. As a concrete example, the Rust programming language supports a single mutable borrow and many `const borrows` [10](§4.2). However, the quantum setting induces additional challenges: only guaranteeing read-only access to variables is insufficient as we must also ensure safe uncomputation. To this end, Silq supports a combination of **const** and **qfree**.

Qfree. To our knowledge, no existing quantum language annotates **qfree** functions. ReVerC’s language fragment contains **qfree** functions (e.g., **X**), and ReQWire’s syntactic conditions cover some **qfree** operations, but neither language explicitly introduces or annotates **qfree** functions.

Mfree. Of the languages in Tab. 2, only Q# can prevent reversing measurement and conditioning measurement (via special annotations). However, as Q# cannot detect implicit measurements, reverse and conditionals may still induce unexpected semantics. For other languages, reversal may fail at runtime when reversing measurements, and control may fail at runtime on conditional measurement.

We note that QWire’s **reverse** returns safe functions, but only when given unitary functions (otherwise, it reports a runtime error by outputting `None`). Thus, it for example cannot reverse **dup**, which is linearly isometric but not unitary.

Semantics. The semantics of Silq is conceptually inspired by Selinger and Valiron, who describe an operational semantics of a lambda calculus that operates on a separate quantum storage [28]. However, as a key difference, Silq’s semantics is more intuitive due to automatic uncomputation.

All other languages in Tab. 2 support semantics in terms of circuits that are dynamically constructed by the program.

10 Conclusion

We presented Silq, a new high-level statically typed quantum programming language which ensures safe uncomputation. This enables an intuitive semantics that is physically realizable on a QRAM.

Our evaluation shows that quantum algorithms expressed in Silq are significantly more concise and less cluttered compared to their version in other quantum languages.

References

- [1] Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, Francisco Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, Jerry M. Chow, Antonio D. Córcoles-Gonzales, Abigail J. Cross, Andrew Cross, Juan Cruz-Benito, Chris Culver, Salvador De La Puente González, Enrique De La Torre, Delton Ding, Eugene Dumitrescu, Ivan Duran, Pieter Eendebak, Mark Everitt, Ismael Faro Sertage, Albert Frisch, Andreas Fuhrer, Jay Gambetta, Borja Godoy Gago, Juan Gomez-Mosquera, Donny Greenberg, Ikko Hamamura, Vojtech Havlicek, Joe Hellmers, Łukasz Herok, Hiroshi Horii, Shaohan Hu, Takashi Imamichi, Toshinari Itoko, Ali Javadi-Abhari, Naoki Kanazawa, Anton Karazeev, Kevin Krsulich, Peng Liu, Yang Luh, Yunho Maeng, Manoel Marques, Francisco Jose Martín-Fernández, Douglas T. McClure, David McKay, Srujan Meesala, Antonio Mezzacapo, Nikolaj Moll, Diego Moreda Rodríguez, Giacomo Nannicini, Paul Nation, Pauline Ollitrault, Lee James O’Riordan, Hanhee Paik, Jesús Pérez, Anna Phan, Marco Pistoia, Viktor Prutyaynov, Max Reuter, Julia Rice, Abdón Rodríguez Davila, Raymond Harry Putra Rudy, Mingi Ryu, Ninad Sathaye, Chris Schnabel, Eddie Schoute, Kanav Setia, Yunong Shi, Adenilton Silva, Yukio Siraichi, Seyon Sivarajah, John A. Smolin, Mathias Soeken, Hitomi Takahashi, Ivano Tavernelli, Charles Taylor, Pete Taylour, Kenso Trabing, Matthew Treinish, Wes Turner, Desiree Vogt-Lee, Christophe Vuillot, Jonathan A. Wildstrom, Jessica Wilson, Erick Winston, Christopher Wood, Stephen Wood, Stefan Wörner, Ismail Yunus Akhalwaya, and Christa Zoufal. 2019. Qiskit: An Open-source Framework for Quantum Computing. <https://doi.org/10.5281/zenodo.2562110>
- [2] Matthew Amy, Martin Roetteler, and Krysta M. Svore. 2017. Verified Compilation of Space-Efficient Reversible Circuits. In *CAV’17*. Vol. 10427. Cham, 3–21. https://doi.org/10.1007/978-3-319-63390-9_1
- [3] Charles H Bennett. 1973. Logical Reversibility of Computation. *IBM Journal of Research and Development* 17, 6 (Nov. 1973), 525–532. <https://doi.org/10.1147/rd.176.0525>
- [4] Andrew M. Childs and Robin Kothari. 2011. Quantum query complexity of minor-closed graph properties. In *STACS’11 (Dagstuhl, Germany, 2011) (Leibniz International Proceedings in Informatics (LIPIcs))*, Vol. 9. 661–672. <https://doi.org/10.4230/LIPIcs.STACS.2011.661>
- [5] Vedran Dunjko, Jacob M. Taylor, and Hans J. Briegel. 2016. Quantum-Enhanced Machine Learning. *Physical Review Letters* 117, 13 (Sept. 2016). <https://doi.org/10.1103/physrevlett.117.130501>
- [6] Timon Gehr, Sasa Misailovic, and Martin Vechev. 2016. Psi: Exact symbolic inference for probabilistic programs. In *CAV’16*. 62–83.
- [7] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: a scalable quantum programming language. In *PLDI’13*. ACM Press, Seattle, Washington, USA. <https://doi.org/10.1145/2491956.2462177>
- [8] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. ACM, 212–219.
- [9] Aram W. Harrow, Avinandan Hassidim, and Seth Lloyd. 2009. Quantum Algorithm for Linear Systems of Equations. *Phys. Rev. Lett.* 103 (Oct 2009), 150502. Issue 15. <https://doi.org/10.1103/PhysRevLett.103.150502>
- [10] Steve Klabnik and Carol Nichols. 2018. *The Rust programming language*. No Starch Press, San Francisco.
- [11] Emmanuel Knill. 1996. *Conventions for quantum pseudocode*. Technical Report. Citeseer.
- [12] Seth Lloyd, Masoud Mohseni, and Patrick Rebentrost. 2014. Quantum principal component analysis. *Nature Physics* 10, 9 (July 2014), 631–633. <https://doi.org/10.1038/nphys3029>
- [13] Guang Hao Low, Theodore J. Yoder, and Isaac L. Chuang. 2014. Quantum inference on Bayesian networks. *Phys. Rev. A* 89, 6 (June 2014), 062315. <https://doi.org/10.1103/PhysRevA.89.062315>
- [14] F. Magniez, M. Santha, and M. Szegedy. 2007. Quantum Algorithms for the Triangle Problem. 37, 2 (2007), 413–424. <https://doi.org/10.1137/050643684>
- [15] Microsoft. 2018. Public submissions of the Microsoft Q# Coding Contest - Summer 2018. (2018). <https://codeforces.com/contest/1002/>
- [16] Microsoft. 2019. Public submissions of the Microsoft Q# Coding Contest - Winter 2019. (2019). <https://codeforces.com/contest/1116/>
- [17] Mariia Mykhailova and Martin Roetteler. 2018. Microsoft Q# Coding Contest - Summer 2018 - Main Contest July 6–9, 2018. (2018). <https://assets.codeforces.com/rounds/997-998/main-contest-editorial.pdf>
- [18] Mariia Mykhailova and Martin Roetteler. 2019. Microsoft Q# Coding Contest - Winter 2019 - Main Contest March 1–4, 2019. (2019). <https://assets.codeforces.com/rounds/1116/contest-editorial.pdf>
- [19] Peter Müller and Arnd Poetzsch-Heffter. 1999. Universes: a type system for controlling representation exposure.
- [20] Daniel Nagaj, Or Sattath, Aharon Brodutch, and Dominique Unruh. 2016. An Adaptive Attack on Wiesner’s Quantum Money. *Quantum Info. Comput.* 16, 11–12 (Sept. 2016), 1048–1070. <http://dl.acm.org/citation.cfm?id=3179330.3179337>
- [21] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum computation and quantum information* (10th anniversary ed ed.). Cambridge University Press, Cambridge ; New York.
- [22] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: a core language for quantum circuits. In *POPL’17*. ACM Press, Paris, France. <https://doi.org/10.1145/3009837.3009894>
- [23] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (Jan. 2019), 299–312. <https://doi.org/10.4204/EPTCS.287.17> arXiv: 1901.10118.
- [24] Patrick Rebentrost, M Mohseni, and Seth Lloyd. 2013. Quantum Support Vector Machine for Big Data Classification. *Physical Review Letters* 113 (2013).
- [25] Steven Roman. 2008. *Advanced Linear Algebra*. New York, NY. <http://site.ebrary.com/id/10230315> OCLC: 730328666.
- [26] Neil J. Ross and Peter LeFanu Lumsdaine. 2015. Algorithms.TF.Main. <https://www.mathstat.dal.ca/~selinger/quipper/doc/Algorithms-TF-Main.html>
- [27] Peter Selinger. 2004. Towards a Quantum Programming Language. *Mathematical Structures in Comp. Sci.* 14, 4 (Aug. 2004), 527–586. <https://doi.org/10.1017/S0960129504004256>
- [28] Peter Selinger and Benoît Valiron. 2006. A lambda calculus for quantum computation with classical control. *Mathematical Structures in Computer Science* 16, 03 (June 2006), 527. <https://doi.org/10.1017/S0960129506005238>
- [29] Damian S. Steiger, Thomas Häner, and Matthias Troyer. 2018. ProjectQ: an open source software framework for quantum computing. *Quantum* 2 (Jan. 2018), 49. <https://doi.org/10.22331/q-2018-01-31-49>
- [30] Krysta Svore, Martin Roetteler, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, and Andres Paz. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 on - RWDSL2018*. ACM Press, Vienna, Austria. <https://doi.org/10.1145/3183895.3183901>
- [31] Google AI Quantum Team. 2017. Cirq. (2017). <https://github.com/google/quantumlib/Cirq>
- [32] Dave Wecker and Krysta M Svore. 2014. LIQ|i>: A software design architecture and domain-specific language for quantum computing. *arXiv preprint arXiv:1402.4467* (2014).
- [33] Nathan Wiebe, Daniel Braun, and Seth Lloyd. 2012. Quantum Algorithm for Data Fitting. *Physical Review Letters* 109, 5 (Aug. 2012). <https://doi.org/10.1103/physrevlett.109.050505>
- [34] Stephen Wiesner. 1983. Conjugate Coding. *SIGACT News* 15, 1 (Jan. 1983), 78–88. <https://doi.org/10.1145/1008908.1008920>